pgTAP & Supabase RLS test suite: generate copy-paste SQL assertions for all 10 OWASP ASVS controls with automated CI runs

Computer
AppSec debugging runbook: 10 student-app vulnerabilities mapped to Jest unit tests and GitHub Actions block rules
Computer
Show me the pgTAP test file for AC 03
Write the Jest test file for authentication tokens
Draft the GitHub Actions workflow file for tests

Generated a copy-paste-ready **pgTAP, Jest, and GitHub Actions security suite** for Semester. It includes the requested AC-03 RLS test, authentication-token Jest tests, a CI workflow, and a 10-vulnerability test/block-policy map.

Files created

File	Place it at	Purpose
`20261006_ac03_cross_tenant_rls.test.sql`	`supabase/tests/`	pgTAP test for AC-03: T1 user cannot read, update, or insert T2 data
`authTokens.test.ts`	`app/src/security/`	Jest/Vitest-compatible JWT verification tests
`security-tests.yml`	`.github/workflows/`	Security CI workflow: unit tests, SAST, dependency audit, secret scan, local Supabase, pgTAP
`APPSEC-10-VULNERABILITIES-TEST-CI-MAP.md`	`docs/assurance/`	Ten student-app vulnerabilities mapped to tests and merge blockers
`semester-pgtap-jest-ci-suite.zip`	—	Bundle with all four files

AC-03 pgTAP test

The generated test file uses the correct RLS pattern:

1. Seed synthetic T1 and T2 fixture records using a privileged **setup-only** role.
2. Confirm both fixtures exist before any RLS assertion.

3. Switch to `authenticated`.
4. Set the simulated JWT claims for Student A in T1.
5. Confirm Student A can read their permitted T1 record.
6. Confirm they cannot read, update, or insert a T2 record.
7. Return to the setup verifier and prove the forbidden mutation did not occur.
8. Roll back all synthetic changes.

Key assertion pattern:

```
set local role authenticated;

select set_config(
  'request.jwt.claims',
  '{"sub":"student-a-id","role":"authenticated"}',
  true
);

select is_empty(
  $$ select 1
     from app.student_records
     where id = 'record-owned-by-t2' $$,
  'student A cannot read T2 record'
);

select results_eq(
  $$ with attempted as (
       update app.student_records
       set display_name = 'blocked-cross-tenant-write'
       where id = 'record-owned-by-t2'
       returning id
     )
     select count(*) from attempted $$,
  $ values (0::bigint) $,
  'student A cannot update T2 record'
);
```

The positive T1 assertion is essential. Without it, a “deny” test can falsely pass because fixtures are missing or the policy denies every user. Supabase supports pgTAP database testing and documents role/JWT-context testing for RLS behavior.[1] [2]

Before running, replace the placeholder schema/table names, UUIDs, and fixture-procedure comments with the confirmed Semester schema. The file must run with `authenticated` for enforcement assertions—never `postgres` or `service_role`.

Authentication Jest test

The generated `authTokens.test.ts` uses `jose` and an ephemeral unit-test key. It validates all of the critical token checks:

- Valid token accepted.

- Expired token rejected.
- Wrong issuer rejected.
- Wrong audience rejected.
- Missing subject rejected.
- Invalid application role rejected.
- Token signed by another key rejected.

Example:

```
const { payload } = await jwtVerify(token, key, {
  algorithms: ["HS256"],
  issuer,
  audience,
  currentDate: now,
  maxTokenAge: "60m",
  clockTolerance: 0,
});
```

JWT validation should explicitly constrain accepted algorithms, issuer, audience, expiration, signature key, and application-specific claims. The `jose` verifier supports accepted algorithms plus issuer, audience, current date, and maximum-token-age checks. [3]

Replace the test-only HS256 secret with your real production verification mechanism—typically a verified JWKS/public key or trusted Supabase auth configuration. Do not reuse the test secret or add production tokens to source code.

GitHub Actions workflow

The workflow has four jobs:

Job	Blocks merge when
`appsec-unit`	Type check, Jest/Vitest tests, SAST, or dependency policy fails
`secrets`	Gitleaks finds a secret or an action uses mutable `vX`, `main`, `master`, or `latest` tag
`rls-policy`	Local Supabase migration/reset or pgTAP policy tests fail
`merge-gate`	Any required security job fails

The workflow triggers on pull requests that touch `app/`, `supabase/`, workflow files, or Node dependency files, as well as pushes to `main`.

Before committing it, replace:

```
REPLACE_WITH_REVIEWED_COMMIT_SHA
REPLACE_WITH_REVIEWED_DIGEST
REPLACE_WITH_REVIEWED_SUPABASE_CLI_VERSION
```

Also confirm that your repository defines these scripts:

```
npm run typecheck
npm run test:ci
npm run security:sast
supabase test db
```

Supabase supports GitHub Actions with its CLI, and the official setup action installs the CLI on hosted runners. [4] [5]

Ten vulnerability block rules

The runbook maps these risks to automated controls:

1. SQL injection → Jest validation tests and SAST.
2. XSS → React rendering tests and SAST.
3. RLS bypass / IDOR → pgTAP AC-03 plus direct API tests.
4. Role escalation → pgTAP protected-field/grant tests.
5. Token or session flaws → JWT Jest tests plus revocation tests.
6. File/export IDOR → API and storage policy tests.
7. Unsafe uploads → upload validation and storage policy tests.
8. CSRF/CORS failures → browser/API state-change tests.
9. SSRF → URL allowlist and redirect validation tests.
10. Secret/CI supply-chain risk → Gitleaks, dependency audit, and immutable action pinning.

Branch protection must independently mark `appsec-unit`, `secrets`, `rls-policy`, and at least one code review as required. A workflow file reports failures, but branch protection is what prevents those failures from being merged.

⁂

1. https://supabase.com/docs/guides/local-development/testing/pgtap-extended
2. https://supabase.com/docs/guides/database/extensions
3. https://github.com/panva/jose/blob/main/docs/jwt/verify/interfaces/JWTVerifyOptions.md
4. https://supabase.com/docs/guides/functions/examples/github-actions
5. https://github.com/supabase/setup-cli
6. https://github.com/marketplace/actions/supabase-cli-action
7. https://stackoverflow.com/questions/77992218/github-actions-failing-after-upgrading-to-node-v20
8. https://stackoverflow.com/questions/66823618/is-options-ignoreexpiration-still-valid-in-node-jose
9. https://supabase.com/docs/guides/local-development/testing/overview
10. https://supabase.com/docs/guides/database/extensions/pgtap
11. https://makerkit.dev/docs/next-supabase-turbo/development/database-tests
12. https://hexdocs.pm/gose/2.0.0/gose/jose/jwt.html
13. https://pgtap.org/documentation.html

14. https://connect2id.com/products/nimbus-jose-jwt/examples/validating-jwt-access-tokens

15. https://github.com/actions/setup-node